Advanced Agent Engineering — 강사용 상세 대본

- 슬라이드 수: 283

- 부제: RAG · HITL · MCP · 평가 · 멀티에이전트 운영

- 기준: source-repo/05_advanced/*.ipynb

---

01. AI 에이전트 개발 고급

안녕하세요. AI 에이전트 개발 고급 과정을 시작하겠습니다. 이번 트랙의 중심은 미들웨어, 멀티에이전트, 메모리, RAG, SQL, 데이터 분석, 음성, 평가와 배포를 운영 관점에서 결합하는 일입니다. 슬라이드는 개념의 흐름을 잡기 위한 자료이고, 뒤쪽 코드 부록은 실제 노트북의 실행 셀을 순서대로 확인하기 위한 자료입니다. 강의 중에는 API 이름을 외우는 것보다, 그 API가 어떤 문제를 해결하려고 등장했는지에 집중해 주세요. 이 기준을 잡으면 이후 실습에서 코드가 길어져도 실행 흐름을 놓치지 않습니다.

---

02. 오리엔테이션

이제 PART 01 오리엔테이션으로 넘어갑니다. LangChain 기초, LangGraph 심화, Deep Agents 실전 위에 미들웨어·멀티에이전트·메모리·RAG·SQL·데이터 분석·음성·프로덕션을 얹습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다.

---

03. 고급 트랙은 에이전트를 기능 구현에서 운영 가능한 시스템으로 끌어올립니다

고급 트랙은 에이전트를 기능 구현에서 운영 가능한 시스템으로 끌어올립니다. 목표는 새로운 API를 많이 아는 것이 아니라 실패·권한·상태·평가를 설계하는 역량입니다. Middleware · Multi-agent · Context/Memory · Agentic RAG · SQL/Data/Voice · Production. 이 덱은 고급 트랙 노트북 10개를 따라가는 학습 안내서입니다. 이 문장은 과정 전체의 기준입니다. 모델이 답을 잘 만들었는지만 보지 않고, 실행 흐름이 통제 가능한지와 문제가 생겼을 때 복구 가능한지를 함께 보겠습니다.

---

04. 05_advanced는 10개 노트북으로 고급 운영 패턴을 묶습니다

05_advanced는 10개 노트북으로 고급 운영 패턴을 묶습니다. 각 노트북은 하나의 운영 문제를 다룹니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 마이그레이션은 v1 패키지·API·상태 스키마 전환입니다. 판단 기준은 기존 코드를 최신 구조로 옮깁니다. 미들웨어는 6단계 훅과 wrap_model/tool_call입니다. 판단 기준은 실행 경계를 제어합니다. 멀티에이전트는 Subagents, Handoffs, Router입니다. 판단 기준은 작업 분리와 상태 전환을 설계합니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

05. 고급 트랙의 공통 축은 실행 흐름을 제어하고 상태를 회수 가능하게 만드는 것입니다

고급 트랙의 공통 축은 실행 흐름을 제어하고 상태를 회수 가능하게 만드는 것입니다. 모든 노트북은 모델 답변이 아니라 실행 경계와 운영 루프를 설명합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Control은 미들웨어와 그래프로 실행 흐름을 제어합니다. State는 체크포인터, Store, runtime context로 정보를 나눕니다. Evaluation은 테스트와 평가로 출시 기준을 검증합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

06. v1 마이그레이션

이제 PART 02 v1 마이그레이션으로 넘어갑니다. 레거시 LangChain 코드를 v1 패키지 구조, create_agent, TypedDict state, runtime context, middleware 방식으로 옮깁니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/00_migration.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

07. 마이그레이션 실습은 import, agent 생성, state, middleware를 순서대로 검증합니다

마이그레이션 실습은 import, agent 생성, state, middleware를 순서대로 검증합니다. 한 번에 전부 바꾸지 말고 실행 표면별로 통과 여부를 확인합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 import는 langchain-classic 의존 위치 확인입니다. 판단 기준은 deprecated 경로가 남지 않습니다. agent 생성은 create_agent로 모델·도구 연결입니다. 판단 기준은 기존 질문이 같은 도구를 호출합니다. state는 TypedDict 기반 상태 정리입니다. 판단 기준은 그래프와 타입 오류가 없습니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

08. 마이그레이션 리스크는 조용한 행동 변화에서 발생합니다

마이그레이션 리스크는 조용한 행동 변화에서 발생합니다. 코드가 실행돼도 도구 순서나 출력 계약이 바뀌면 운영 회귀입니다. 묶음별로 읽으면 주의할 회귀에서는 도구 호출 이름이 바뀌어 기존 평가가 실패합니다; context 전달 위치가 바뀌어 사용자 권한이 누락됩니다; Pydantic state를 그대로 두어 LangGraph 호환성이 떨어집니다를 봅니다. 완료 기준에서는 대표 질문 세트가 이전과 같은 행동을 보입니다; trace에서 도구 호출과 오류 처리가 확인됩니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

09. 마이그레이션은 import 변경보다 실행 모델 변경을 먼저 봐야 합니다

마이그레이션은 import 변경보다 실행 모델 변경을 먼저 봐야 합니다. 패키지 이름만 바꾸면 코드가 돌아가도 운영 기능을 놓칠 수 있습니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 패키지는 langchain-classic 분리입니다. 판단 기준은 레거시 chain/agent 경로를 확인합니다. Agent API는 create_react_agent 중심에서 create_agent 중심으로 이동입니다. 판단 기준은 도구·모델·미들웨어 결합 표면이 바뀝니다. State는 Pydantic state보다 TypedDict state를 사용입니다. 판단 기준은 LangGraph 호환성을 확보합니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

10. create_agent는 모델·도구·미들웨어를 한 실행 표면에서 묶습니다

create_agent는 모델·도구·미들웨어를 한 실행 표면에서 묶습니다. 고급 트랙에서는 create_agent를 기본 조립 단위로 보고 LangGraph/Deep Agents로 확장합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 model은 모델 객체 또는 모델 문자열을 전달합니다. tools는 도구는 함수 또는 Tool 객체로 연결합니다. middleware는 동적 프롬프트, HITL, 오류 처리 등을 삽입합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 agents에서 create_agent를 준비합니다. 핵심 API는 create_agent, invoke입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 노트북 기준 위치는 00_migration.ipynb create_agent입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

11. 마이그레이션 검수는 동작 여부보다 의미 보존 여부를 봐야 합니다

마이그레이션 검수는 동작 여부보다 의미 보존 여부를 봐야 합니다. 기존 코드가 실행되는 것과 기존 권한·상태·출력 계약이 유지되는 것은 다릅니다. 묶음별로 읽으면 코드 검수에서는 import 경로와 deprecated API를 확인합니다; state schema가 TypedDict 기준으로 정리됐는지 봅니다; configurable 값이 context_schema로 옮겨졌는지 봅니다를 봅니다. 행동 검수에서는 도구 호출 순서가 유지되는지 확인합니다; 오류 처리와 재시도 정책이 middleware로 이동했는지 봅니다; 테스트 질문에서 이전 답변 계약이 유지되는지 검증합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

12. 미들웨어

이제 PART 03 미들웨어로 넘어갑니다. 모델 호출 전후, 도구 호출 전후, 에이전트 전후 훅으로 실행 흐름을 제어합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/01_middleware.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

13. 미들웨어 실습은 훅 위치와 반환 타입을 함께 확인해야 합니다

미들웨어 실습은 훅 위치와 반환 타입을 함께 확인해야 합니다. 훅은 실행 위치에 따라 허용되는 입력과 반환이 다릅니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 before_agent는 입력 검증과 초기 상태입니다. 판단 기준은 차단 또는 상태 업데이트가 명확합니다. wrap_model_call은 모델 호출 감싸기입니다. 판단 기준은 handler 호출 전후 정책이 분리됩니다. wrap_tool_call은 도구 오류와 권한입니다. 판단 기준은 ToolMessage 또는 Command 처리가 맞습니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

14. 미들웨어 리스크는 너무 많은 정책을 한 훅에 넣을 때 커집니다

미들웨어 리스크는 너무 많은 정책을 한 훅에 넣을 때 커집니다. 정책을 섞으면 실패 원인과 테스트 범위가 불분명해집니다. 묶음별로 읽으면 분리 원칙에서는 입력 차단은 before_agent에 둡니다; 모델 선택은 wrap_model_call에 둡니다; 도구 오류 복구는 wrap_tool_call에 둡니다를 봅니다. 테스트 기준에서는 허용 케이스와 차단 케이스를 모두 둡니다; 예외가 사용자에게 그대로 노출되지 않아야 합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

15. 미들웨어는 에이전트 실행 파이프라인의 각 지점에 정책을 삽입합니다

미들웨어는 에이전트 실행 파이프라인의 각 지점에 정책을 삽입합니다. 고급 에이전트는 프롬프트보다 미들웨어로 운영 정책을 관리하는 경우가 많습니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 before_agent는 입력 검증과 초기 컨텍스트 조립을 수행합니다. wrap_model_call은 모델 호출을 감싸 모델·프롬프트·응답을 제어합니다. wrap_tool_call은 도구 오류, 재시도, 권한 확인을 처리합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

16. 6단계 훅은 정책을 어디에 넣을지 정하는 지도입니다

6단계 훅은 정책을 어디에 넣을지 정하는 지도입니다. 같은 정책도 위치가 다르면 의미가 달라집니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 before_agent는 에이전트 실행 전입니다. 판단 기준은 입력 검증, PII 차단입니다. before_model은 모델 호출 전입니다. 판단 기준은 동적 프롬프트, 컨텍스트 압축입니다.

wrap_model_call은 모델 호출 감싸기입니다. 판단 기준은 모델 라우팅, fallback, 비용 제어입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

17. wrap_tool_call은 도구 오류를 모델이 읽을 수 있는 관측값으로 바꿉니다

wrap_tool_call은 도구 오류를 모델이 읽을 수 있는 관측값으로 바꿉니다. 예외를 그대로 터뜨리면 대화가 끊기지만 ToolMessage로 바꾸면 에이전트가 복구합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 화면에서 request는 도구 호출 이름과 인자를 포함합니다. handler은 실제 도구 실행을 호출합니다. ToolMessage는 오류를 도구 관측값으로 반환합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 middleware에서 wrap_tool_call; messages에서 ToolMessage를 준비합니다. 데코레이터 wrap_tool_call가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 handle_tool_errors는 실제 작업 단위입니다. 노트북 기준 위치는 01_middleware.ipynb wrap_tool_call입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

18. HumanInTheLoopMiddleware는 위험 도구를 실행 전에 멈춥니다

HumanInTheLoopMiddleware는 위험 도구를 실행 전에 멈춥니다. 승인·수정·거절이 필요한 도구는 모델 판단만으로 실행하지 않습니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 approve는 도구 호출을 그대로 실행입니다. 판단 기준은 낮은 위험의 확인 절차입니다. edit는 인자를 사람이 수정해 실행입니다. 판단 기준은 파일 경로, SQL 조건, 수신자 수정입니다. reject는 도구 호출을 실행하지 않음입니다. 판단 기준은 위험 요청 차단입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

19. Subagents

이제 PART 04 Subagents로 넘어갑니다. 전문 에이전트를 도구처럼 호출해 컨텍스트를 격리하고 역할별 도구 권한을 제한합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/02_multi_agent_subagents.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

---

20. Subagents 실습은 역할 경계와 권한 경계를 같이 검수합니다

Subagents 실습은 역할 경계와 권한 경계를 같이 검수합니다. 전문 역할을 나눴다면 호출 조건과 도구 범위도 달라야 합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 description은 호출 조건 문장입니다. 판단 기준은 감독자가 역할을 안정적으로 고릅니다. tools는 역할별 최소 도구입니다. 판단 기준은 권한이 불필요하게 넓지 않습니다. output는 하위 결과 형식입니다. 판단 기준은 감독자가 짧게 합성합니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

21. Subagents 리스크는 위임이 아니라 결과 회수에서 드러납니다

Subagents 리스크는 위임이 아니라 결과 회수에서 드러납니다. 하위 작업 결과가 길거나 모호하면 메인 컨텍스트가 다시 부풀어 오릅니다. 묶음별로 읽으면 좋은 회수에서는 핵심 결론과 출처만 반환합니다; 다음 행동이 필요한 경우 명시합니다를 봅니다. 나쁜 회수에서는 원문 전체를 그대로 반환합니다; 하위 에이전트가 사용자에게 직접 말하는 톤을 유지합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

22. Subagents는 감독자가 전문 에이전트를 도구처럼 호출하는 중앙 집중 패턴입니다

Subagents는 감독자가 전문 에이전트를 도구처럼 호출하는 중앙 집중 패턴입니다. 사용자는 감독자와 대화하고, 전문 작업은 별도 컨텍스트에서 처리됩니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Supervisor은 라우팅과 최종 응답을 책임집니다. Subagent는 전문 도구와 프롬프트만 가지고 독립 작업합니다. Context isolation은 긴 하위 작업이 메인 대화를 오염시키지 않습니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

23. 서브에이전트는 역할 분리와 컨텍스트 격리를 동시에 제공합니다

서브에이전트는 역할 분리와 컨텍스트 격리를 동시에 제공합니다. 역할을 많이 만드는 것보다 description과 도구 권한을 선명하게 하는 것이 중요합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 name는 도구 이름처럼 노출되는 식별자입니다. 판단 기준은 짧고 명확해야 합니다. description은 언제 호출할지 판단하는 라우팅 문장입니다. 판단 기준은 가장 중요한 필드입니다. system_prompt는 하위 역할의 행동 계약입니다. 판단 기준은 출력 형식과 금지 행동을 넣습니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

24. SubAgentMiddleware는 supervisor에게 task 디스패치 도구를 노출합니다

SubAgentMiddleware는 supervisor에게 task 디스패치 도구를 노출합니다. 직접 @tool로 감싸는 방식보다 역할 목록을 선언적으로 관리합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 subagents는 역할 정의 목록입니다. middleware는 감독자 에이전트에 위임 도구를 주입합니다. task는 모델이 하위 역할에게 작업을 보냅니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 agents에서 create_agent; middleware에서 SubAgentMiddleware를 준비합니다. 핵심 API는 create_agent입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 노트북 기준 위치는 02_multi_agent_subagents.ipynb SubAgentMiddleware입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

25. 서브에이전트 설계의 실패는 대부분 역할 경계가 흐릴 때 생깁니다

서브에이전트 설계의 실패는 대부분 역할 경계가 흐릴 때 생깁니다. 각 역할이 같은 도구와 같은 출력 형식을 가지면 분리 효과가 줄어듭니다. 묶음별로 읽으면 좋은 분리에서는 역할마다 입력과 출력이 다릅니다; 도구 권한이 역할별로 제한됩니다; 감독자는 결과 요약만 받습니다를 봅니다. 나쁜 분리에서는 모든 서브에이전트가 같은 일을 합니다; description이 모호해 라우팅이 흔들립니다; 하위 결과가 너무 길어 메인 컨텍스트를 다시 채웁니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

26. Handoffs와 Router

이제 PART 05 Handoffs와 Router로 넘어갑니다. 상태 기반 순차 전환과 분류 기반 병렬 라우팅을 구분합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/03_multi_agent_handoffs_router.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

27. Handoffs와 Router 실습은 전이 메시지와 fan-out 결과를 확인합니다

Handoffs와 Router 실습은 전이 메시지와 fan-out 결과를 확인합니다. 멀티에이전트 오류는 대부분 상태 전이 또는 결과 합성에서 발생합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Handoff ToolMessage는 tool_call_id 매칭입니다. 판단 기준은 대화 히스토리가 malformed 되지 않습니다. current_step는 상태 전이 값입니다. 판단 기준은 단계별 프롬프트와 도구가 바뀝니다. Send fan-out는 워커 호출 목록입니다. 판단 기준은 필요한 도메인만 병렬 실행됩니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

28. 패턴 선택 리스크는 순차 업무를 병렬화하거나 병렬 검색을 순차화할 때 생깁니다

패턴 선택 리스크는 순차 업무를 병렬화하거나 병렬 검색을 순차화할 때 생깁니다. 제어 흐름이 업무 흐름과 맞지 않으면 성능보다 사용자 경험이 먼저 무너집니다. 묶음별로 읽으면 Handoffs가 맞는 경우에서는 사용자와 단계별로 직접 대화합니다; 앞 단계 정보가 뒤 단계의 전제입니다를 봅니다. Router가 맞는 경우에서는 여러 지식 소스를 동시에 검색합니다; 분류 결과에 따라 필요한 워커 수가 달라집니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

29. Handoffs는 단일 에이전트가 상태에 따라 다른 페르소나와 도구를 갖는 패턴입니다

Handoffs는 단일 에이전트가 상태에 따라 다른 페르소나와 도구를 갖는 패턴입니다. 고객 지원처럼 순차 단계가 있는 업무는 current_step 상태로 진행 단계를 관리합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 current_step는 현재 단계를 추적하는 상태 변수입니다. Command(update)는 핸드오프 도구가 상태를 바꿉니다. wrap_model_call은 단계별 system prompt와 tools를 교체합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

30. 핸드오프 도구는 Command(update)와 ToolMessage를 함께 반환해야 합니다

핸드오프 도구는 Command(update)와 ToolMessage를 함께 반환해야 합니다. 도구 호출/응답 쌍이 맞지 않으면 모델 대화 히스토리가 잘못된 형태가 됩니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 화면에서 runtime는 tool_call_id를 얻습니다. current_step는 다음 단계로 상태를 전환합니다. ToolMessage는 핸드오프 ack를 메시지에 남깁니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 tools에서 tool, ToolRuntime; types에서 Command를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 transfer_to_resolve는 실제 작업 단위입니다. 노트북 기준 위치는 03_multi_agent_handoffs_router.ipynb handoff tool입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

31. Router는 분류 후 Send로 여러 도메인에 fan-out하고 reducer에서 합성합니다

Router는 분류 후 Send로 여러 도메인에 fan-out하고 reducer에서 합성합니다. 병렬 지식 검색과 결과 합성이 필요한 경우 Handoffs보다 Router가 적합합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Classify는 질문을 도메인 또는 작업 유형으로 분류합니다. Send는 필요한 워커를 병렬로 실행합니다. Reducer은 여러 결과를 출처와 함께 합성합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

32. Subagents, Handoffs, Router는 제어 주체가 다릅니다

Subagents, Handoffs, Router는 제어 주체가 다릅니다. 멀티에이전트 패턴은 에이전트 수가 아니라 제어 흐름으로 구분합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Subagents는 감독자가 제어입니다. 판단 기준은 전문 작업 위임입니다. Handoffs는 상태가 제어입니다. 판단 기준은 순차 단계와 사용자 직접 대화입니다. Router은 분류 결과가 제어입니다. 판단 기준은 병렬 검색과 fan-out입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

33. 컨텍스트와 메모리

이제 PART 06 컨텍스트와 메모리로 넘어갑니다. runtime context, store, semantic memory, progressive disclosure로 정보 수명과 가변성을 분리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/04_context_memory.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

34. 컨텍스트 실습은 정보의 수명과 가변성을 먼저 분류합니다

컨텍스트 실습은 정보의 수명과 가변성을 먼저 분류합니다. 정보가 어디에 둘지 정해야 비용과 보안 경계가 보입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Runtime context는 실행 중 고정 정보입니다. 판단 기준은 권한·사용자 id가 도구에 주입됩니다. State는 현재 실행 중 변화 정보입니다. 판단 기준은 노드 사이 값이 추적됩니다. Store는 스레드 밖 장기 정보입니다. 판단 기준은 다음 대화에서도 검색됩니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

35. 메모리 리스크는 오래 보관할 필요 없는 정보를 장기 저장할 때 커집니다

메모리 리스크는 오래 보관할 필요 없는 정보를 장기 저장할 때 커집니다. 장기 메모리는 편리하지만 삭제, 권한, 개인정보 책임을 함께 가집니다. 묶음별로 읽으면 저장 전 질문에서는 다음 대화에서도 필요한 정보인가요?; 사용자가 삭제를 요청맡길 일인가요?; 다른 thread나 사용자에게 보이면 안 되나요?를 봅니다. 운영 기준에서는 namespace로 사용자와 조직을 분리합니다; 민감정보는 저장 전 마스킹하거나 저장하지 않습니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

36. 컨텍스트는 가변성과 수명을 기준으로 나눠야 합니다

컨텍스트는 가변성과 수명을 기준으로 나눠야 합니다. 무엇을 prompt에 넣을지보다 어디에 얼마나 오래 보관할지를 먼저 결정합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Static runtime context는 실행 중 바뀌지 않는 설정입니다. 판단 기준은 사용자 id, 권한, 조직 정보입니다. Dynamic state는 그래프 실행 중 바뀌는 상태입니다. 판단 기준은 messages, 중간 결과입니다. Cross-thread store는 스레드 밖에서 공유되는 장기 정보입니다. 판단 기준은 사용자 프로필, 장기 메모리입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

37. context_schema는 런타임 정보를 타입 있는 객체로 주입합니다

context_schema는 런타임 정보를 타입 있는 객체로 주입합니다. 도구와 미들웨어가 같은 사용자·권한 정보를 일관되게 읽게 만듭니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 화면에서 dataclass는 런타임 컨텍스트 스키마입니다. context_schema는 agent가 받을 context 타입을 지정합니다. runtime.context는 도구 실행 중 컨텍스트를 읽습니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 dataclasses에

서 dataclass; tools에서 ToolRuntime, tool를 준비합니다. 클래스 UserContext는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다. 노트북 기준 위치는 04_context_memory.ipynb context_schema입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

38. Store는 thread를 넘어 공유되는 장기 메모리를 저장합니다

Store는 thread를 넘어 공유되는 장기 메모리를 저장합니다. checkpointer가 실행 이력을 저장한다면 Store는 사용자나 조직 단위의 장기 지식을 저장합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 화면에서 namespace는 사용자나 도메인별 저장 공간입니다. put는 장기 사실을 저장합니다. search는 필요한 기억을 검색합니다. 노트북 기준 위치는 04_context_memory.ipynb Store memory입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

39. 컨텍스트 엔지니어링은 많이 넣는 기술이 아니라 필요한 정보만 회수하는 기술입니다

컨텍스트 엔지니어링은 많이 넣는 기술이 아니라 필요한 정보만 회수하는 기술입니다. 정보를 잘못된 층에 넣으면 비용, 누락, 권한 문제가 동시에 생깁니다. 묶음별로 읽으면 prompt에 넣을 것에서는 이번 답변에 반드시 필요한 짧은 지침; 모델이 바로 따라야 하는 출력 계약를 봅니다 store에 넣을 것에서는 다음 대화에서도 필요한 장기 사실; 사용자별로 검색 가능한 선호와 히스토리를 봅니다 state에 넣을 것에서는 현재 실행 중만 필요한 중간 결과; 그래프 노드 사이에 전달되는 작업 상태를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

---

40. Agentic RAG

이제 PART 07 Agentic RAG로 넘어갑니다. 검색 결과를 평가하고 질문을 다시 쓰는 그래프형 RAG를 구축합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/05_agentic_rag.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

---

41. Agentic RAG 실습은 검색 품질과 복구 경로를 따로 검증합니다

Agentic RAG 실습은 검색 품질과 복구 경로를 따로 검증합니다. 최종 답변만 보면 검색 실패와 생성 실패를 구분할 수 없습니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 검색 결과는 정답 문서가 상위에 있는지입니다. 판단 기준은 top-k 안에 기대 근거가 있습니다. 관련성 평가는 grade_documents 결과입니다. 판단 기준은 무관한 문서는 답변으로 가지 않습니다. 질문 재작성은 rewrite_question 효과입니다. 판단 기준은 재검색 결과가 개선됩니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

42. Agentic RAG 리스크는 재시도가 품질 개선이 아니라 루프가 될 때 발생합니다

Agentic RAG 리스크는 재시도가 품질 개선이 아니라 루프가 될 때 발생합니다. 복구 경로에는 항상 횟수 제한과 실패 응답 정책이 필요합니다. 묶음별로 읽으면 제한할 것에서는 질문 재작성 횟수; 검색 top-k와 컨텍스트 길이; 모델 호출 총량를 봅니다. 실패 시 응답에서는 근거 부족을 명시합니다; 추가로 필요한 문서나 질문을 제안합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

43. Agentic RAG는 검색 후 관련성 평가와 질문 재작성을 포함합니다

Agentic RAG는 검색 후 관련성 평가와 질문 재작성을 포함합니다. 단순 RAG가 실패하는 지점을 그래프의 복구 경로로 명시합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Retrieve는 질문에 맞는 문서 후보를 가져옵니다. Grade는 문서가 질문에 답가능한지 평가합니다. Rewrite는 관련성이 낮으면 질문을 검색 친화적으로 다시 씁니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

44. retrieve 도구는 content와 artifact를 함께 반환해야 평가와 출처 추적이 가능합니다

retrieve 도구는 content와 artifact를 함께 반환해야 평가와 출처 추적이 가능합니다. 모델이 읽을 문자열과 운영자가 추적할 Document 객체를 분리합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 화면에서 serialized는 모델 입력에 넣는 근거 문자열입니다. docs는 원본 Document 객체입니다. metadata는 출처 표시와 필터링에 사용합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 tools에서 tool를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 retrieve는 실제 작업 단위입니다. 노트북 기준 위치는 05_agentic_rag.ipynb retrieve tool입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

45. relevance_router는 관련성 평가 결과를 답변 또는 재검색 경로로 바꿉니다

relevance_router는 관련성 평가 결과를 답변 또는 재검색 경로로 바꿉니다. 그래프형 RAG의 핵심은 검색 실패를 최종 답변 전에 감지하는 것입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 화면에서 relevant는 답변 생성으로 이동합니다. not relevant는 질문 재작성으로 이동합니다. StateGraph는 복구 경로를 실행 그래프에 고정합니다. 코드는 다음 지점을 중심으로 읽습니다. 함수 relevance_router는 실제 작업 단위입니다. 핵심 API는 StateGraph, add_node입니다. 노트북 기준 위치는 05_agentic_rag.ipynb relevance routing입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

46. 세 가지 RAG 접근은 제어 가능성과 비용이 다릅니다

세 가지 RAG 접근은 제어 가능성과 비용이 다릅니다. 복잡한 질문일수록 복구 경로가 필요하지만 호출 비용과 지연이 늘어납니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 RAG Chain은 검색 후 답변 흐름 고정입니다. 판단 기준은 단순 질의응답입니다. RAG Agent는 도구 사용 여부를 모델이 판단입니다. 판단 기준은 다단계 검색 질문입니다. LangGraph RAG는 평가·재작성·답변을 그래프로 고정입니다. 판단 기준은 운영 복구와 관찰 가능성입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

47. SQL Agent

이제 PART 08 SQL Agent로 넘어갑니다. 자연어 질문을 안전한 SQL 조회와 검증 가능한 답변으로 연결합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/06_sql_agent.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

48. SQL Agent 실습은 생성 SQL을 최종 답변만큼 중요하게 다룹니다

SQL Agent 실습은 생성 SQL을 최종 답변만큼 중요하게 다룹니다. 데이터 기반 답변은 SQL과 결과를 함께 검토해야 합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 schema 조회는 사용 가능한 테이블 확인입니다. 판단 기준은 없는 컬럼을 쓰지 않습니다. query 생성은 SELECT와 조건 구성입니다. 판단 기준은 허용 테이블과 LIMIT를 지킵니다. query 실행은 결과 행과 오류 확인입니다. 판단 기준은 오류를 수정 루프로 보냅니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

49. SQL Agent 리스크는 자연어가 권한 높은 쿼리로 바뀌는 순간 발생합니다

SQL Agent 리스크는 자연어가 권한 높은 쿼리로 바뀌는 순간 발생합니다. 읽기 전용이어도 민감 데이터 대량 조회는 운영 사고가 됩니다. 묶음별로 읽으면 차단할 요청에서는 개인정보 전체 추출; 권한 밖 테이블 조회; 쓰기·삭제·스키마 변경를 봅니다. 완화 방법에서는 허용 테이블과 행 수를 제한합니다; 위험 쿼리는 HITL 승인을 둡니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

50. SQLDatabaseToolkit은 DB 스키마 조회와 SQL 실행 도구를 에이전트에 제공합니다

SQLDatabaseToolkit은 DB 스키마 조회와 SQL 실행 도구를 에이전트에 제공합니다. SQL 에이전트는 먼저 스키마를 읽고, 쿼리를 작성하고, 결과를 해석해야 합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 SQLDatabase는 연결 대상 DB와 테이블 정보를 제공합니다. Toolkit는 스키마 조회와 쿼리 실행 도구를 만듭니다. Agent는 도구를 사용해 자연어 질문에 답합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 utilities에서 SQLDatabase; agent_toolkits에서 SQLDatabaseToolkit를 준비합니다. 핵심 API는 create_agent입니다. 노트북 기준 위치는 06_sql_agent.ipynb SQL toolkit입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

51. SQL 에이전트 안전 수칙은 조회 범위와 실행 권한을 제한하는 데서 시작합니다

SQL 에이전트 안전 수칙은 조회 범위와 실행 권한을 제한하는 데서 시작합니다. 자연어가 SQL이 되는 순간 데이터 삭제와 대량 조회 위험을 함께 봐야 합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 읽기 전용은 SELECT 중심 권한 사용입니다. 판단 기준은 쓰기·삭제 차단입니다. 테이블 제한은 허용 테이블만 노출입니다. 판단 기준은 민감 테이블 보호입니다. 쿼리 검토는 실행 전 SQL 확인입니다. 판단 기준은 HITL 승인입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

52. SQL Agent도 위험한 실행 전에는 HITL 경계를 둘 수 있습니다

SQL Agent도 위험한 실행 전에는 HITL 경계를 둘 수 있습니다. 사용자 질문이 데이터베이스 실행으로 이어지므로 승인과 검토 루프가 중요합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Generate SQL은 모델이 조회 쿼리를 제안합니다. Review는 사람 또는 정책이 위험 쿼리를 검토합니다. Execute는 허용된 쿼리만 실행합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

53. SQL 답변은 쿼리, 결과, 해석을 분리해 검증해야 합니다

SQL 답변은 쿼리, 결과, 해석을 분리해 검증해야 합니다. 최종 한국어 답변만 남기면 어떤 데이터에서 나온 결론인지 추적할 수 없습니다. 묶음별로 읽으면 남겨야 할 것에서는 사용자 질문; 생성된 SQL; 실행 결과 행 수와 샘플를 봅니다. 거절해야 할 것에서는 권한 밖 테이블 조회; 대량 개인정보 추출; 쓰기·삭제·스키마 변경를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

54. 데이터 분석 에이전트

이제 PART 09 데이터 분석 에이전트로 넘어갑니다. Deep Agents와 샌드박스 실행으로 데이터 분석 작업을 산출물 중심으로 수행합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/07_data_analysis.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

55. 데이터 분석 실습은 산출물 파일과 해석 문장을 함께 검수합니다

데이터 분석 실습은 산출물 파일과 해석 문장을 함께 검수합니다. 분석 에이전트는 코드를 실행하므로 결과 재현성이 핵심입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 입력 데이터는 파일과 스키마 확인입니다. 판단 기준은 행 수와 컬럼이 기록됩니다. 분석 코드는 전처리·집계·시각화입니다. 판단 기준은 재실행 가능합니다. 산출물은 표·차트·보고서 파일입니다. 판단 기준은 작업 폴더에 남습니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

56. 데이터 분석 리스크는 그럴듯한 차트가 검증 없이 제시될 때 커집니다

데이터 분석 리스크는 그럴듯한 차트가 검증 없이 제시될 때 커집니다. 에이전트가 만든 분석도 사람이 만든 분석과 같은 검산 절차가 필요합니다. 묶음별로 읽으면 검산 항목에서는 분모와 집계 단위; 결측과 이상치 처리; 필터와 기간 조건를 봅니다. 보고 항목에서는 분석 질문; 사용한 데이터; 결론과 한계를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

57. 데이터 분석 에이전트는 질문·코드·파일·해석이 모두 남아야 합니다

데이터 분석 에이전트는 질문·코드·파일·해석이 모두 남아야 합니다. 분석 에이전트는 답변보다 재현 가능한 작업 흔적이 중요합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 질문은 분석 목표와 지표 정의입니다. 판단 기준은 모호하면 잘못된 코드가 생성됩니다. 코드는 전처리·집계·시각화 실행입니다. 판단 기준은 재현성과 리뷰 대상입니다. 파일은 입력 데이터와 출력 산출물입니다. 판단 기준은 작업 공간 경계가 필요합니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

58. Deep Agents는 파일 백엔드와 도구를 묶어 분석 작업 공간을 만듭니다

Deep Agents로 데이터 분석 작업 공간을 만들 때는 파일 백엔드와 계획 미들웨어를 구분합니다. FilesystemBackend는 분석 파일의 경계를 잡고, TodoListMiddleware는 분석 절차가 길어질 때 단계 관리를 맡습니다. provider별 설정은 별도 회귀 항목으로 분리해 확인합니다.

59. 분석 코드 실행은 에이전트 제어면과 샌드박스를 분리해서 봐야 합니다

분석 코드 실행은 에이전트 제어면과 샌드박스를 분리해서 봐야 합니다. 여기에 provider 설정도 운영 항목으로 붙습니다. OpenAI prompt_cache_options나 Google thinking_config처럼 모델 제공자별 옵션은 분석 로직 자체와 섞지 않고 환경 검증 대상으로 둡니다.

60. 분석 결과는 숫자보다 검증 문장이 먼저 필요합니다

분석 결과는 숫자보다 검증 문장이 먼저 필요합니다. 차트와 표가 있어도 단위, 필터, 결측 기준이 없으면 업무 의사결정에 쓰기 어렵습니다. 묶음별로 읽으면 검증 문장에서는 사용한 데이터 기간과 필터를 밝힙니다; 행 수, 결측, 이상치를 확인합니다; 집계 단위와 분모를 설명합니다를 봅니다. 해석 문장에서는 결론과 근거 지표를 연결합니다; 대체 설명과 한계를 적습니다; 다음 분석 질문을 제안합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

61. Voice Agent

이제 PART 10 Voice Agent로 넘어갑니다. STT, Agent, TTS와 멀티모달 content blocks로 음성 에이전트 아키텍처를 설계합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/08_voice_agent.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

62. Voice Agent 실습은 정확도보다 지연·중단·승인 흐름을 함께 봐야 합니다

Voice Agent 실습은 정확도보다 지연·중단·승인 흐름을 함께 봐야 합니다. 음성 UI에서는 사용자가 기다리는 시간과 끼어들기가 제품 품질을 좌우합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 STT 지연은 음성 인식까지 걸리는 시간입니다. 판단 기준은 사용자가 긴 침묵을 느끼지 않습니다. Agent 지연은 도구 호출과 추론 시간입니다. 판단 기준은 중간 상태를 안내합니다. TTS 제어는 응답 재생과 중단입니다. 판단 기준은 barge-in을 처리합니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

63. Voice Agent 리스크는 오인식과 과잉 실행이 결합될 때 커집니다

Voice Agent 리스크는 오인식과 과잉 실행이 결합될 때 커집니다. 잘못 들은 명령을 곧바로 도구 실행으로 연결하지 않아야 합니다. 묶음별로 읽으면 안전 장치에서는 민감 작업은 다시 읽고 승인받습니다; 사용자 발화를 텍스트로 확인하도록 합니다; 실패 시 텍스트 fallback을 제공합니다를 봅니다. 로그 기준에서는 음성 원문 저장 여부를 정책화합니다; 인식 결과와 실행 명령을 분리해 기록합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

64. Voice Agent는 STT, Agent, TTS가 지연 시간 안에서 이어지는 시스템입니다

Voice Agent는 STT, Agent, TTS가 지연 시간 안에서 이어지는 시스템입니다. 음성은 정확도뿐 아니라 응답 지연과 중간 끊김이 사용자 경험을 좌우합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 STT는 사용자 음성을 텍스트로 변환입니다. 판단 기준은 실시간성, 화자 구분입니다. Agent는 텍스트 질문을 이해하고 도구를 사용입니다. 판단 기준은 대화 상태와 권한입니다. TTS는 응답을 음성으로 변환입니다. 판단 기준은 톤, 속도, 중단 처리입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

65. 음성 에이전트 아키텍처는 세 가지 방식으로 비교합니다

음성 에이전트 아키텍처는 세 가지 방식으로 비교합니다. 어떤 컴포넌트가 실시간 스트림을 잡고 있는지에 따라 구현 난도가 달라집니다. 카드별로 보면 Sandwich는 STT → Agent → TTS를 순서대로 연결합니다. 가장 이해하기 쉽습니다. Realtime는 모델 또는 런타임이 음성 스트림을 직접 다룹니다. 지연이 낮습니다. Hybrid는 중요 도구 호출은 텍스트 에이전트로, 대화 입출력은 음성으로 처리합니다. Fallback는 음성 실패 시 텍스트 UI로 전환합니다. 운영 안정성이 높습니다. 이 슬라이드는 선택지를 나열하는 장면입니다. 실제 설계에서는 지금 보이는 항목 중 무엇을 기본값으로 둘지, 무엇을 나중에 확장할지 결정해야 합니다.

66. 음성 에이전트는 끼어들기와 중단을 제품 요구사항으로 다뤄야 합니다

음성 에이전트는 끼어들기와 중단을 제품 요구사항으로 다뤄야 합니다. 텍스트 채팅과 달리 사용자는 말하는 중간에 멈추거나 수정하거나 말을 끊습니다. 묶음별로 읽으면 대화 제어에서는 사용자 barge-in을 감지합니다; TTS 재생을 중단해야 합니다; 부분 인식 결과와 최종 인식 결과를 구분합니다를 봅니다. 운영 제어에서는 민감 작업은 음성만으로 실행하지 않습니다; 승인 문구를 명확히 다시 읽어 줍니다; 오인식 로그와 재시도 경로를 남깁니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

67. output_version="v1"은 멀티모달 content blocks를 표준 형태로 받기 위한 설정입니다

output_version="v1"은 멀티모달 content blocks를 표준 형태로 받기 위한 설정입니다. 음성·이미지·텍스트가 섞이는 앱에서는 응답 구조를 고정해야 프런트엔드 처리가 단순해집니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 화면에서 TextContentBlock은 일반 텍스트 응답입니다. ImageContentBlock은 이미지 입력 또는 출력과 연결됩니다. Reasoning block는 모델 제공자별 추론 블록을 표준 구조로 다룹니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI, invoke입니다. 노트북 기준 위치는 08_voice_agent.ipynb content blocks입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

68. 프로덕션

이제 PART 11 프로덕션으로 넘어갑니다. 테스트, 평가, 관측성, 배포, 롤백을 에이전트 제품 출시 기준으로 묶습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/09_production.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

69. 프로덕션 실습은 평가셋과 배포 체크리스트가 있어야 완료됩니다

프로덕션 실습은 평가셋과 배포 체크리스트가 있어야 완료됩니다. 기능이 돌아가도 평가와 롤백 기준이 없으면 출시 상태가 아닙니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 평가셋은 대표 질문과 기대 결과입니다. 판단 기준은 릴리즈마다 재실행됩니다. 관측성은 trace와 비용 지표입니다. 판단 기준은 실패 지점이 보입니다. 보안은 권한·PII·HITL 정책입니다. 판단 기준은 감사 로그가 남습니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

70. 프로덕션 리스크는 모델 품질보다 변경 관리 부재에서 자주 발생합니다

프로덕션 리스크는 모델 품질보다 변경 관리 부재에서 자주 발생합니다. 프롬프트 한 줄 변경도 도구 호출 경로와 비용을 바꿉니다. 묶음별로 읽으면 변경 전에서는 회귀 평가셋을 실행합니다; 비용과 지연 변화를 비교합니다; 위험 도구 케이스를 재검증합니다를 봅니다. 변경 후에서는 실패 trace를 데이터셋에 추가합니다; 사용자 피드백을 평가 항목으로 바꿉니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

71. 에이전트 테스트는 Unit, Integration, Evals로 나눠야 합니다

에이전트 테스트는 Unit, Integration, Evals로 나눠야 합니다. 함수 테스트만 통과해도 실제 에이전트 행동은 실패합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Unit는 도구 함수와 라우터 단위 검증입니다. 판단 기준은 결정론적 테스트입니다. Integration은 모델·도구·그래프 연결 검증입니다. 판단 기준은 mock 또는

fake model입니다. Evals는 실제 질문 세트 품질 평가입니다. 판단 기준은 LangSmith dataset입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

72. LangSmith 평가는 질문, 기대 답변, 실행 결과를 데이터셋으로 비교합니다

LangSmith 평가는 질문, 기대 답변, 실행 결과를 데이터셋으로 비교합니다. 프로덕션에서는 한두 번의 수동 시연보다 반복 가능한 평가셋이 필요합니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 화면에서 dataset는 대표 질문과 기대 결과 모음입니다. evaluator은 정답성, 도구 경로, 안전성을 채점합니다. experiment는 모델·프롬프트 변경 전후를 비교합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 langsmith에서 Client를 준비합니다. 노트북 기준 위치는 09_production.ipynb LangSmith eval입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

73. SmithDB 조회는 runs.query로 프로젝트·기간·projection을 명시합니다

SmithDB 조회는 runs.query만 쓰면 끝나는 것이 아닙니다. 공식 migration 기준으로 projection을 명시하지 않으면 기본적으로 ID만 반환됩니다. 운영 분석에서 이름, 시작 시각, 종료 시각, 입력, 출력까지 보려면 uppercase selects에 ID, NAME, START_TIME, END_TIME, INPUTS, OUTPUTS를 명시해야 합니다. 그래서 이 예제는 project_ids, min_start_time, max_start_time, page_size와 함께 selects를 고정합니다.

---

74. LangSmith migration은 runs, feedback, sharing 영역을 따로 판단합니다

LangSmith migration은 runs, feedback, sharing, exceptions를 따로 판단합니다. runs.query에서는 project_ids와 기간, page_size뿐 아니라 어떤 필드를 받을지 selects로 정해야 합니다. selects를 빼면 ID만 받는다는 점을 기억해야 trace 분석, 이름 기반 필터링, 입출력 검토가 헷갈리지 않습니다.

---

75. evaluation tracing은 비용과 디버깅 사이에서 선택합니다

evaluation tracing은 비용과 디버깅 사이의 선택입니다. evaluate 자체는 회귀 평가의 기준 경로로 유지합니다. 다만 evaluator 내부 trace까지 남길지, disable_evaluator_tracing으로 비용을 줄일지는 채점 과정을 얼마나 디버깅해야 하는지에 따라 결정합니다.

---

76. 공식 문서 이동과 신규 항목은 기존 트랙에 연결해 읽습니다

공식 문서 이동과 신규 항목은 기존 트랙에 연결해 읽습니다. Code와 plugin 문서는 통합 트랙으로, dynamic subagents는 서브에이전트 설계로, fault tolerance는 durable execution으로, multimodal과 retrieval은 기존 RAG와 voice, data analysis 실습으로 연결합니다. 새 용어를 나열하는 대신 어느 수업에서 검증할지 남깁니다.

---

77. 운영 모니터링은 답변 품질과 시스템 비용을 함께 봐야 합니다

운영 모니터링은 답변 품질과 시스템 비용을 함께 봐야 합니다. 에이전트는 모델 호출, 도구 호출, 사람 승인, 외부 API 비용이 함께 발생합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 품질은 정답성, 근거성, 거절 정확도입니다. 판단 기준은 평가셋과 사용자 피드백입니다. 경로는 도구 호출 순서와 실패 지점입니다. 판단 기준은 trace와 trajectory입니다. 비용은 토큰, 도구 API, 실행 시간입니다. 판단 기준은 예산과 호출 제한입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

78. 출시 기준은 기능 완료가 아니라 실패 대응 절차가 준비됐는지로 판단합니다

출시 기준은 기능 완료가 아니라 실패 대응 절차가 준비됐는지로 판단합니다. 고급 에이전트는 실패하지 않는 시스템이 아니라 실패를 발견하고 되돌릴 수 있는 시스템이어야 합니다. 묶음별로 읽으면 출시 전에서는 회귀 평가셋을 통과합니다; 민감 도구의 승인 정책을 검증합니다; 비용과 지연 예산을 정합니다를 봅니다. 출시 후에서는 실패 trace를 수집합니다; 오답을 데이터셋에 추가합니다; 프롬프트, 도구, 그래프 변경을 실험 단위로 비교합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

79. 개념 근거와 고급 패턴

이제 REFERENCE 개념 근거와 고급 패턴으로 넘어갑니다. 고급 패턴 설명은 ReAct, RAG, Pregel 논문, Anthropic 에이전트 패턴 문서, LangChain/LangGraph 공식 문서를 기준으로 보강합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 여기서는 개념 설명의 근거를 정리합니다. 실무에서 프레임워크를 설명할 때는 개인적인 비유보다 공식 문서와 논문에 기대는 편이 안전합니다.

80. 고급 에이전트 패턴은 control loop, retrieval loop, evaluation loop로 나눠 설명할 수 있습니다

고급 에이전트 패턴은 control loop, retrieval loop, evaluation loop로 나눠 설명합니다. ReAct는 추론과 행동의 교차 루프를, RAG는 검색과 생성 결합을, Anthropic은 workflow와 agent 패턴의 구분을 제시합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 ReAct는 reasoning과 acting을 번갈아 수행합니다. RAG는 parametric memory와 non-parametric memory를 결합합니다. Evaluator은 생성 결과를 평가하고 개선하는 루프입니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

81. 프로덕션 에이전트는 trace, eval, guardrail, rollback을 한 루프로 관리합니다

프로덕션 에이전트는 trace, eval, guardrail, rollback을 한 루프로 관리합니다. 공식 LangSmith/LangGraph 문서와 Anthropic 패턴은 평가와 관찰 가능성을 운영 루프의 핵심으로 봅니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Trace는 도구 호출과 상태 변화를 기록합니다. Eval은 대표 질문 세트로 회귀를 확인합니다. Rollback은 문제 변경을 되돌려야 합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

82. 고급 개념 보강 출처는 논문과 공식 문서를 함께 사용합니다

고급 개념 보강 출처는 논문과 공식 문서를 함께 사용합니다. 응용별 개념은 가장 직접적인 신뢰 출처를 기준으로 설명합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Tool-using agent는 ReAct, arXiv:2210.03629입니다. 판단 기준은 reason-act-observe 루프와 trajectory입니다. RAG는 Lewis et al., NeurIPS 2020입니다. 판단 기준은 parametric/non-parametric memory와 provenance입니다. Workflow patterns는 Anthropic Building Effective Agents입니다. 판단 기준은 prompt chaining, routing, parallelization, orchestrator-worker, evaluator-optimizer입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

83. 00_migration.ipynb

이제 CODE 00_migration.ipynb로 넘어갑니다. 00_migration.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/00_migration.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름

의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

84. 00_migration 코드 01

이제 00_migration 코드 01 슬라이드입니다. 노트북 위치는 00_migration.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 add는 실제 작업 단위입니다. 핵심 API는 create_agent, ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

85. 00_migration 코드 02

이제 00_migration 코드 02 슬라이드입니다. 노트북 위치는 00_migration.ipynb의 cell 6이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

86. 00_migration 코드 03

이제 00_migration 코드 03 슬라이드입니다. 노트북 위치는 00_migration.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 AgentState, create_agent를 준비합니다. 클래스 CustomState는 데이터 구조나 정책 단위를 정의합니다. 핵심 API는 create_agent, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

87. 00_migration 코드 04 (1/2)

이제 00_migration 코드 04 (1/2) 슬라이드입니다. 노트북 위치는 00_migration.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 dataclasses에서 dataclass; tools에서 tool, ToolRuntime를 준비합니다. 클래스 UserContext는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 get_role는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

88. 00_migration 코드 04 (2/2)

이제 00_migration 코드 04 (2/2) 슬라이드입니다. 노트북 위치는 00_migration.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

89. 00_migration 코드 05

이제 00_migration 코드 05 슬라이드입니다. 노트북 위치는 00_migration.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 dynamic_prompt, ModelRequest를 준비합니다. 데코레이터 dynamic_prompt가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 role_based_prompt는 실제 작업 단위입니다. 핵심 API는 create_agent, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

90. 00_migration 코드 06

이제 00_migration 코드 06 슬라이드입니다. 노트북 위치는 00_migration.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 wrap_tool_call; messages에서 ToolMessage를 준비합니다. 데코레이터 wrap_tool_call가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 handle_errors는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

91. 00_migration 코드 07

이제 00_migration 코드 07 슬라이드입니다. 노트북 위치는 00_migration.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

92. 00_migration 코드 08

이제 00_migration 코드 08 슬라이드입니다. 노트북 위치는 00_migration.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

93. 01_middleware.ipynb

이제 CODE 01_middleware.ipynb로 넘어갑니다. 01_middleware.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/01_middleware.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

94. 01_middleware 코드 01

이제 01_middleware 코드 01 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 3이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

95. 01_middleware 코드 02

이제 01_middleware 코드 02 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

96. 01_middleware 코드 03

이제 01_middleware 코드 03 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 6이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent를 준비합니다. 핵심 API는 create_agent, HumanInTheLoopMiddleware입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

97. 01_middleware 코드 04

이제 01_middleware 코드 04 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 SummarizationMiddleware를 준비합니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

98. 01_middleware 코드 05

이제 01_middleware 코드 05 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 HumanInTheLoopMiddleware; memory에서 InMemorySaver를 준비합니다. 핵심 API는 InMemorySaver, HumanInTheLoopMiddleware입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

99. 01_middleware 코드 06

이제 01_middleware 코드 06 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent, InMemorySaver입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

100. 01_middleware 코드 07

이제 01_middleware 코드 07 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 Command를 준비합니다. 핵심 API는 invoke, Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

101. 01_middleware 코드 08

이제 01_middleware 코드 08 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 ModelCallLimitMiddleware를 준비합니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

102. 01_middleware 코드 09

이제 01_middleware 코드 09 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 ToolCallLimitMiddleware를 준비합니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

103. 01_middleware 코드 10

이제 01_middleware 코드 10 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 ModelFallbackMiddleware를 준비합니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

104. 01_middleware 코드 11

이제 01_middleware 코드 11 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 PIIMiddleware를 준비합니다. 핵심 API는 PIIMiddleware입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

105. 01_middleware 코드 12

이제 01_middleware 코드 12 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 PIIMiddleware입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

106. 01_middleware 코드 13

이제 01_middleware 코드 13 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 re를 준비합니다. 핵심 API는 PIIMiddleware, compile입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

107. 01_middleware 코드 14

이제 01_middleware 코드 14 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 함수 detect_ssn는 실제 작업 단위입니다. 핵심 API는 PIIMiddleware입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

108. 01_middleware 코드 15

이제 01_middleware 코드 15 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 LLMToolSelectorMiddleware를 준비합니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

109. 01_middleware 코드 16

이제 01_middleware 코드 16 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 langchain_anthropic에서 ChatAnthropic; middleware에서 AnthropicPromptCachingMiddleware를 준비합니다. 핵심 API는 create_agent입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

110. 01_middleware 코드 17

이제 01_middleware 코드 17 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 27이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 langchain_aws에서 ChatBedrockConverse; prompt_caching에서 BedrockPromptCachingMiddleware를 준비합니다. 핵심 API는 create_agent입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

111. 01_middleware 코드 18

이제 01_middleware 코드 18 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 28이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 patch_tool_calls에서 PatchToolCallsMiddleware를 준비합니다. 핵심 API는 create_agent, create_deep_agent입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

112. 01_middleware 코드 19

이제 01_middleware 코드 19 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 29이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 ContextEditingMiddleware, ClearToolUsesEdit를 준비합니다. 핵심 API는 create_agent입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

113. 01_middleware 코드 20

이제 01_middleware 코드 20 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 31이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 before_model를 준비합니다. 데코레이터 before_model가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 log_before는 실제 작업 단위입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

114. 01_middleware 코드 21

이제 01_middleware 코드 21 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 32이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 after_model를 준비합니다. 데코레이터 after_model가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 validate_output는 실제 작업 단위입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

115. 01_middleware 코드 22

이제 01_middleware 코드 22 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 33이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Callable; middleware에서 wrap_model_call, ModelRequest, ModelResponse를 준비합니다. 데코레이터 wrap_model_call가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 retry_on_error는 실제 작업 단위입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

116. 01_middleware 코드 23

이제 01_middleware 코드 23 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 34이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Callable; middleware에서 wrap_tool_call, ToolCallRequest를 준비합니다. 데코레이터 wrap_tool_call가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 audit_tool는 실제 작업 단위입니다. 핵심 API는 Command입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

117. 01_middleware 코드 24

이제 01_middleware 코드 24 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 35이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 wrap_model_call가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 dynamic_config는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

118. 01_middleware 코드 25

이제 01_middleware 코드 25 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 36이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Any; middleware에서 after_model, hook_config, AgentState를 준비합니다. 데코레이터 after_model, hook_config가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 block_forbidden는 실제 작업 단위입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

119. 01_middleware 코드 26

이제 01_middleware 코드 26 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 37이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 ExtendedModelResponse를 준비합니다. 데코레이터 wrap_model_call가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 track_tokens는 실제 작업 단위입니다. 핵심 API는 Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

120. 01_middleware 코드 27

이제 01_middleware 코드 27 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 38이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 AgentMiddleware를 준비합니다. 클래스 AuditMiddleware는 데이터 구조나 정책 단위를 정의합니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

121. 01_middleware 코드 28

이제 01_middleware 코드 28 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 40이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 데코레이터 before_model, before_model, before_model가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 mw_a, mw_b, mw_c는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

122. 01_middleware 코드 29

이제 01_middleware 코드 29 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 42이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent; patch_tool_calls에서 PatchToolCallsMiddleware를 준비합니다. 핵심 API는 create_agent, InMemorySaver, HumanInTheLoopMiddleware, PIIMiddleware입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

123. 01_middleware 코드 30

이제 01_middleware 코드 30 슬라이드입니다. 노트북 위치는 01_middleware.ipynb의 cell 43이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent, InMemorySaver, PIIMiddleware입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

124. 02_multi_agent_subagents.ipynb

이제 CODE 02_multi_agent_subagents.ipynb로 넘어갑니다. 02_multi_agent_subagents.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/02_multi_agent_subagents.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

125. 02_multi_agent_subagents 코드 01

이제 02_multi_agent_subagents 코드 01 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 3이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

126. 02_multi_agent_subagents 코드 02

이제 02_multi_agent_subagents 코드 02 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

127. 02_multi_agent_subagents 코드 03

이제 02_multi_agent_subagents 코드 03 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 create_calendar_event는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

128. 02_multi_agent_subagents 코드 04

이제 02_multi_agent_subagents 코드 04 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 read_calendar_events는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

129. 02_multi_agent_subagents 코드 05

이제 02_multi_agent_subagents 코드 05 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 send_email, read_emails는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

130. 02_multi_agent_subagents 코드 06

이제 02_multi_agent_subagents 코드 06 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 search_emails는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

131. 02_multi_agent_subagents 코드 07

이제 02_multi_agent_subagents 코드 07 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent를 준비합니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

132. 02_multi_agent_subagents 코드 08

이제 02_multi_agent_subagents 코드 08 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

133. 02_multi_agent_subagents 코드 09

이제 02_multi_agent_subagents 코드 09 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 call_calendar는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

134. 02_multi_agent_subagents 코드 10

이제 02_multi_agent_subagents 코드 10 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 call_email는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

135. 02_multi_agent_subagents 코드 11

이제 02_multi_agent_subagents 코드 11 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

136. 02_multi_agent_subagents 코드 12

이제 02_multi_agent_subagents 코드 12 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

137. 02_multi_agent_subagents 코드 13

이제 02_multi_agent_subagents 코드 13 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 HumanInTheLoopMiddleware; memory에서 InMemorySaver를 준비합니다. 핵심 API는 InMemorySaver, HumanInTheLoopMiddleware입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

138. 02_multi_agent_subagents 코드 14

이제 02_multi_agent_subagents 코드 14 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent, InMemorySaver입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

139. 02_multi_agent_subagents 코드 15

이제 02_multi_agent_subagents 코드 15 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 Command를 준비합니다. 핵심 API는 invoke, Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

140. 02_multi_agent_subagents 코드 16

이제 02_multi_agent_subagents 코드 16 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 dataclasses에서 dataclass; tools에서 tool, ToolRuntime를 준비합니다. 클래스 UserContext, SupervisorState는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

141. 02_multi_agent_subagents 코드 17

이제 02_multi_agent_subagents 코드 17 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 27이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

142. 02_multi_agent_subagents 코드 18

이제 02_multi_agent_subagents 코드 18 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 28이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하

는 실행 단위로 바꿉니다 함수 send_email_ctx는 실제 작업 단위입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

143. 02_multi_agent_subagents 코드 19 (1/2)

이제 02_multi_agent_subagents 코드 19 (1/2) 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 30이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 datetime에서 datetime; uuid를 준비합니다. 데코레이터 tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 start_async_task, check_async_task는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

144. 02_multi_agent_subagents 코드 19 (2/2)

이제 02_multi_agent_subagents 코드 19 (2/2) 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 30이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

145. 02_multi_agent_subagents 코드 20 (1/2)

이제 02_multi_agent_subagents 코드 20 (1/2) 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 31이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool, tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 update_async_task, cancel_async_task, list_async_tasks는 실제 작업 단위입니다. 핵심 API는 interrupt입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

146. 02_multi_agent_subagents 코드 20 (2/2)

이제 02_multi_agent_subagents 코드 20 (2/2) 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 31이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

147. 02_multi_agent_subagents 코드 21

이제 02_multi_agent_subagents 코드 21 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 33이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Literal를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 dispatch는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

148. 02_multi_agent_subagents 코드 22

이제 02_multi_agent_subagents 코드 22 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 34이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

149. 02_multi_agent_subagents 코드 23 (1/2)

이제 02_multi_agent_subagents 코드 23 (1/2) 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 35이고, 화면에는 실제 실행 코드 중 21줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 subagents에서 SubAgentMiddleware를 준비합니다. 핵심 API는 create_agent입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

150. 02_multi_agent_subagents 코드 23 (2/2)

이제 02_multi_agent_subagents 코드 23 (2/2) 슬라이드입니다. 노트북 위치는 02_multi_agent_subagents.ipynb의 cell 35이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

151. 03_multi_agent_handoffs_router.ipynb

이제 CODE 03_multi_agent_handoffs_router.ipynb로 넘어갑니다. 03_multi_agent_handoffs_router.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/03_multi_agent_handoffs_router.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

152. 03_multi_agent_handoffs_router 코드 01

이제 03_multi_agent_handoffs_router 코드 01 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

153. 03_multi_agent_handoffs_router 코드 02

이제 03_multi_agent_handoffs_router 코드 02 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다.

운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

154. 03_multi_agent_handoffs_router 코드 03

이제 03_multi_agent_handoffs_router 코드 03 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 AgentState; typing에서 Literal를 준비합니다. 클래스 SupportState는 데이터 구조나 정책 단위를 정의합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

155. 03_multi_agent_handoffs_router 코드 04

이제 03_multi_agent_handoffs_router 코드 04 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool; types에서 Command를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 lookup_customer는 실제 작업 단위입니다. 핵심 API는 Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

156. 03_multi_agent_handoffs_router 코드 05

이제 03_multi_agent_handoffs_router 코드 05 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 check_service_status는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

157. 03_multi_agent_handoffs_router 코드 06

이제 03_multi_agent_handoffs_router 코드 06 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 escalate_to_resolve는 실제 작업 단위입니다. 핵심 API는 Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

158. 03_multi_agent_handoffs_router 코드 07

이제 03_multi_agent_handoffs_router 코드 07 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 apply_fix는 실제 작업 단위입니다. 핵심 API는 Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

159. 03_multi_agent_handoffs_router 코드 08

이제 03_multi_agent_handoffs_router 코드 08 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 mark_resolved는 실제 작업 단위입니다. 핵심 API는 Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

160. 03_multi_agent_handoffs_router 코드 09

이제 03_multi_agent_handoffs_router 코드 09 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 ToolRuntime; messages에서 ToolMessage를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 transfer_to_resolve는 실제 작업 단위입니다. 핵심 API는 Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

161. 03_multi_agent_handoffs_router 코드 10

이제 03_multi_agent_handoffs_router 코드 10 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 send_satisfaction_survey, close_ticket는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

162. 03_multi_agent_handoffs_router 코드 11

이제 03_multi_agent_handoffs_router 코드 11 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

163. 03_multi_agent_handoffs_router 코드 12

이제 03_multi_agent_handoffs_router 코드 12 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

164. 03_multi_agent_handoffs_router 코드 13

이제 03_multi_agent_handoffs_router 코드 13 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Callable; middleware에서 wrap_model_call, ModelRequest, ModelResponse를 준비합

니다. 데코레이터 wrap_model_call가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 step_middleware는 실제 작업 단위입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

165. 03_multi_agent_handoffs_router 코드 14

이제 03_multi_agent_handoffs_router 코드 14 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent를 준비합니다. 핵심 API는 create_agent입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

166. 03_multi_agent_handoffs_router 코드 15

이제 03_multi_agent_handoffs_router 코드 15 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

167. 03_multi_agent_handoffs_router 코드 16

이제 03_multi_agent_handoffs_router 코드 16 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 27이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 pydantic에서 BaseModel, Field; typing에서 Literal를 준비합니다. 클래스 SubQuery, QueryClassification는 데이터 구조나 정책 단위를 정의합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

168. 03_multi_agent_handoffs_router 코드 17

이제 03_multi_agent_handoffs_router 코드 17 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 28이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 AgentState를 준비합니다. 클래스 RouterState는 데이터 구조나 정책 단위를 정의합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

169. 03_multi_agent_handoffs_router 코드 18

이제 03_multi_agent_handoffs_router 코드 18 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 30이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 langchain_openai에서 ChatOpenAI를 준비합니다. 함수 route_query는 실제 작업 단위입니다. 핵심 API는 ChatOpenAI, invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

170. 03_multi_agent_handoffs_router 코드 19

이제 03_multi_agent_handoffs_router 코드 19 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 32이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool; agents에서 create_agent를 준비합니다. 데코레이터 tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 search_github_code, search_notion_pages는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

171. 03_multi_agent_handoffs_router 코드 20

이제 03_multi_agent_handoffs_router 코드 20 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 33이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 search_slack_messages는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

172. 03_multi_agent_handoffs_router 코드 21

이제 03_multi_agent_handoffs_router 코드 21 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 34이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

173. 03_multi_agent_handoffs_router 코드 22

이제 03_multi_agent_handoffs_router 코드 22 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 35이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

174. 03_multi_agent_handoffs_router 코드 23

이제 03_multi_agent_handoffs_router 코드 23 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 36이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 Command, Send를 준비합니다. 함수 dispatch_to_agents는 실제 작업 단위입니다. 핵심 API는 Command, Send입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

175. 03_multi_agent_handoffs_router 코드 24

이제 03_multi_agent_handoffs_router 코드 24 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 38이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는

이렇습니다. 함수 reduce_results는 실제 작업 단위입니다. 핵심 API는 ChatOpenAI, invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

176. 03_multi_agent_handoffs_router 코드 25

이제 03_multi_agent_handoffs_router 코드 25 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 39이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END를 준비합니다. 핵심 API는 StateGraph, add_node입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

177. 03_multi_agent_handoffs_router 코드 26

이제 03_multi_agent_handoffs_router 코드 26 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 40이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 compile, add_edge, add_conditional_edges입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

178. 03_multi_agent_handoffs_router 코드 27

이제 03_multi_agent_handoffs_router 코드 27 슬라이드입니다. 노트북 위치는 03_multi_agent_handoffs_router.ipynb의 cell 41이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

179. 04_context_memory.ipynb

이제 CODE 04_context_memory.ipynb로 넘어갑니다. 04_context_memory.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/04_context_memory.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

180. 04_context_memory 코드 01

이제 04_context_memory 코드 01 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI, OpenAIEmbeddings를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

181. 04_context_memory 코드 02

이제 04_context_memory 코드 02 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

182. 04_context_memory 코드 03

이제 04_context_memory 코드 03 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 dataclasses에서 dataclass; tools에서 tool, ToolRuntime를 준비합니다. 클래스 UserContext는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

183. 04_context_memory 코드 04

이제 04_context_memory 코드 04 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 get_permissions는 실제 작업 단위입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

184. 04_context_memory 코드 05

이제 04_context_memory 코드 05 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent, invoke입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

185. 04_context_memory 코드 06

이제 04_context_memory 코드 06 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 검색 결과를 모델 답변의 근거로 연결하는 흐름을 다룹니다. 검색, 문서 구성, 출처 보존을 함께 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 AgentState를 준비합니다. 클래스 RAGState는 데이터 구조나 정책 단위를 정의합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

186. 04_context_memory 코드 07

이제 04_context_memory 코드 07 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent, invoke입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

187. 04_context_memory 코드 08

이제 04_context_memory 코드 08 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 whoami는 실제 작업 단위입니다. 설정값 thread_id는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

188. 04_context_memory 코드 09

이제 04_context_memory 코드 09 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 memory에서 InMemoryStore를 준비합니다. 핵심 API는 InMemoryStore입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

189. 04_context_memory 코드 10

이제 04_context_memory 코드 10 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

190. 04_context_memory 코드 11

이제 04_context_memory 코드 11 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 InMemoryStore입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

191. 04_context_memory 코드 12

이제 04_context_memory 코드 12 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

192. 04_context_memory 코드 13

이제 04_context_memory 코드 13 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

193. 04_context_memory 코드 14

이제 04_context_memory 코드 14 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 27이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 get_user_info는 실제 작업 단위입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

194. 04_context_memory 코드 15

이제 04_context_memory 코드 15 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 28이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 save_preference는 실제 작업 단위입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

195. 04_context_memory 코드 16

이제 04_context_memory 코드 16 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 29이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent, invoke입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

196. 04_context_memory 코드 17

이제 04_context_memory 코드 17 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 30이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 typing_extensions에서 TypedDict를 준비합니다. 클래스 UserInfo는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 save_user_info, get_user_info_typed는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

197. 04_context_memory 코드 18

이제 04_context_memory 코드 18 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 32이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 InMemoryStore입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

198. 04_context_memory 코드 19

이제 04_context_memory 코드 19 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 33이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

199. 04_context_memory 코드 20

이제 04_context_memory 코드 20 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 34이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

200. 04_context_memory 코드 21

이제 04_context_memory 코드 21 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 36이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

201. 04_context_memory 코드 22

이제 04_context_memory 코드 22 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 37이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 load_skill는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

202. 04_context_memory 코드 23

이제 04_context_memory 코드 23 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 40이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 base에서 BaseStore를 준비합니다. 함수 reflect_node는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

203. 04_context_memory 코드 24

이제 04_context_memory 코드 24 슬라이드입니다. 노트북 위치는 04_context_memory.ipynb의 cell 41이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 asyncio를 준비합니다. 함수 background_memory_writer는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

204. 05_agentic_rag.ipynb

이제 CODE 05_agentic_rag.ipynb로 넘어갑니다. 05_agentic_rag.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/05_agentic_rag.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

205. 05_agentic_rag 코드 01

이제 05_agentic_rag 코드 01 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI, OpenAIEmbeddings를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

206. 05_agentic_rag 코드 02

이제 05_agentic_rag 코드 02 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

207. 05_agentic_rag 코드 03

이제 05_agentic_rag 코드 03 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 검색 결과를 모델 답변의 근거로 연결하는 흐름을 다룹니다. 검색, 문서 구성, 출처 보존을 함께 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 langchain_text_splitters에서 RecursiveCharacterTextSplitter; documents에서 Document를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

208. 05_agentic_rag 코드 04

이제 05_agentic_rag 코드 04 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

209. 05_agentic_rag 코드 05

이제 05_agentic_rag 코드 05 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 검색 결과를 모델 답변의 근거로 연결하는 흐름을 다룹니다. 검색, 문서 구성, 출처 보존을 함께 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 vectorstores에서 InMemoryVectorStore를 준비합니다. print 출력은 학습

용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

210. 05_agentic_rag 코드 06

이제 05_agentic_rag 코드 06 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 retrieve는 실제 작업 단위입니다. 설정값 response_format는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

211. 05_agentic_rag 코드 07

이제 05_agentic_rag 코드 07 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent를 준비합니다. 핵심 API는 create_agent, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

212. 05_agentic_rag 코드 08

이제 05_agentic_rag 코드 08 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 dynamic_prompt를 준비합니다. 데코레이터 dynamic_prompt가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 rag_prompt는 실제 작업 단위입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

213. 05_agentic_rag 코드 09

이제 05_agentic_rag 코드 09 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent, invoke입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

214. 05_agentic_rag 코드 10

이제 05_agentic_rag 코드 10 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 MessagesState를 준비합니다. 클래스 AgentState는 데이터 구조나 정책 단위를 정의합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

215. 05_agentic_rag 코드 11

이제 05_agentic_rag 코드 11 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 검색 결과를 모델 답변의 근거로 연결하는 흐름을 다룹니다. 검색, 문서 구성, 출처 보존을 함께 봅니다. 읽는 순서는 이렇습니다. 함수 generate_query_or_respond는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

216. 05_agentic_rag 코드 12

이제 05_agentic_rag 코드 12 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 검색 결과를 모델 답변의 근거로 연결하는 흐름을 다룹니다. 검색, 문서 구성, 출처 보존을 함께 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 pydantic에서 BaseModel, Field; typing에서 Literal를 준비합니다. 클래스 GradeDocuments는 데이터 구조나 정책 단위를 정의합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

217. 05_agentic_rag 코드 13 (1/2)

이제 05_agentic_rag 코드 13 (1/2) 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 25이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 함수 grade_documents는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

218. 05_agentic_rag 코드 13 (2/2)

이제 05_agentic_rag 코드 13 (2/2) 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 25이고, 화면에는 실제 실행 코드 중 1줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

219. 05_agentic_rag 코드 14 (1/2)

이제 05_agentic_rag 코드 14 (1/2) 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 27이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 rewrite_question는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

220. 05_agentic_rag 코드 14 (2/2)

이제 05_agentic_rag 코드 14 (2/2) 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 27이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

221. 05_agentic_rag 코드 15 (1/2)

이제 05_agentic_rag 코드 15 (1/2) 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 29이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 generate_answer는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

222. 05_agentic_rag 코드 15 (2/2)

이제 05_agentic_rag 코드 15 (2/2) 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 29이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

223. 05_agentic_rag 코드 16

이제 05_agentic_rag 코드 16 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 31이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; prebuilt에서 ToolNode, tools_condition를 준비합니다. 함수 relevance_router는 실제 작업 단위입니다. 핵심 API는 StateGraph, add_node입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

224. 05_agentic_rag 코드 17

이제 05_agentic_rag 코드 17 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 32이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 핵심 API는 add_node, add_edge, add_conditional_edges입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

225. 05_agentic_rag 코드 18

이제 05_agentic_rag 코드 18 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 33이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 검색 결과를 모델 답변의 근거로 연결하는 흐름을 다룹니다. 검색, 문서 구성, 출처 보존을 함께 봅니다. 읽는 순서는 이렇습니다. 핵심 API는 compile, add_edge, add_conditional_edges입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

226. 05_agentic_rag 코드 19

이제 05_agentic_rag 코드 19 슬라이드입니다. 노트북 위치는 05_agentic_rag.ipynb의 cell 34이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

227. 06_sql_agent.ipynb

이제 CODE 06_sql_agent.ipynb로 넘어갑니다. 06_sql_agent.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/06_sql_agent.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

228. 06_sql_agent 코드 01

이제 06_sql_agent 코드 01 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

229. 06_sql_agent 코드 02

이제 06_sql_agent 코드 02 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

230. 06_sql_agent 코드 03

이제 06_sql_agent 코드 03 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 agent_toolkits에서 SQLDatabaseToolkit를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

231. 06_sql_agent 코드 04

이제 06_sql_agent 코드 04 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

232. 06_sql_agent 코드 05

이제 06_sql_agent 코드 05 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent를 준비합니다. 핵심 API는 create_agent입니다.

print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

233. 06_sql_agent 코드 06

이제 06_sql_agent 코드 06 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

234. 06_sql_agent 코드 07

이제 06_sql_agent 코드 07 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 HumanInTheLoopMiddleware를 준비합니다. 핵심 API는 create_agent, HumanInTheLoopMiddleware입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

235. 06_sql_agent 코드 08 (1/2)

이제 06_sql_agent 코드 08 (1/2) 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 Command를 준비합니다. 핵심 API는 invoke, interrupt, Command입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

236. 06_sql_agent 코드 08 (2/2)

이제 06_sql_agent 코드 08 (2/2) 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke, Command입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

237. 06_sql_agent 코드 09

이제 06_sql_agent 코드 09 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Annotated; typing_extensions에서 TypedDict를 준비합니다. 클래스 SQLState는 데이터 구조나 정책 단위를 정의합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

238. 06_sql_agent 코드 10

이제 06_sql_agent 코드 10 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 함수 list_tables_node는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

239. 06_sql_agent 코드 11

이제 06_sql_agent 코드 11 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 get_schema_node는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

240. 06_sql_agent 코드 12

이제 06_sql_agent 코드 12 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 함수 generate_query_node는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

241. 06_sql_agent 코드 13

이제 06_sql_agent 코드 13 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 check_query_node는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

242. 06_sql_agent 코드 14 (1/2)

이제 06_sql_agent 코드 14 (1/2) 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 interrupt를 준비합니다. 함수 execute_query_node는 실제 작업 단위입니다. 핵심 API는 invoke, interrupt입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

243. 06_sql_agent 코드 14 (2/2)

이제 06_sql_agent 코드 14 (2/2) 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

244. 06_sql_agent 코드 15

이제 06_sql_agent 코드 15 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 28이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; memory에서 InMemorySaver를 준비합니다. 핵심 API는 StateGraph, InMemorySaver, add_node입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

245. 06_sql_agent 코드 16

이제 06_sql_agent 코드 16 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 29이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 InMemorySaver, compile, add_edge입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

246. 06_sql_agent 코드 17

이제 06_sql_agent 코드 17 슬라이드입니다. 노트북 위치는 06_sql_agent.ipynb의 cell 30이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 Command를 준비합니다. 핵심 API는 invoke, interrupt, Command입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

247. 07_data_analysis.ipynb

이제 CODE 07_data_analysis.ipynb로 넘어갑니다. 07_data_analysis.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/07_data_analysis.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

248. 07_data_analysis 코드 01

이제 07_data_analysis 코드 01 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 2줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv를 준비합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

249. 07_data_analysis 코드 02

이제 07_data_analysis 코드 02 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream

입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

250. 07_data_analysis 코드 03

이제 07_data_analysis 코드 03 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 backends에서 LocalShellBackend를 준비합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

251. 07_data_analysis 코드 04

이제 07_data_analysis 코드 04 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

252. 07_data_analysis 코드 05

이제 07_data_analysis 코드 05 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

253. 07_data_analysis 코드 06

이제 07_data_analysis 코드 06 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 slack_send_message는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

254. 07_data_analysis 코드 07

이제 07_data_analysis 코드 07 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 tavily에서 TavilyClient를 준비합니다. 함수 tavily_search는 실제 작업 단위입니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

255. 07_data_analysis 코드 08

이제 07_data_analysis 코드 08 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 deepagents에서 create_deep_agent; backends에서 LocalShellBackend를 준비합니다. 핵심 API는 create_deep_agent, InMemorySaver입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

256. 07_data_analysis 코드 09

이제 07_data_analysis 코드 09 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 핵심 API는 create_deep_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

257. 07_data_analysis 코드 10

이제 07_data_analysis 코드 10 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

258. 07_data_analysis 코드 11

이제 07_data_analysis 코드 11 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

259. 07_data_analysis 코드 12

이제 07_data_analysis 코드 12 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

260. 07_data_analysis 코드 13

이제 07_data_analysis 코드 13 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 memory에서 InMemorySaver를 준비합니다. 핵심 API는 create_deep_agent, InMemorySaver입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

261. 07_data_analysis 코드 14

이제 07_data_analysis 코드 14 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 27이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

262. 07_data_analysis 코드 15

이제 07_data_analysis 코드 15 슬라이드입니다. 노트북 위치는 07_data_analysis.ipynb의 cell 28이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

263. 08_voice_agent.ipynb

이제 CODE 08_voice_agent.ipynb로 넘어갑니다. 08_voice_agent.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/08_voice_agent.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

264. 08_voice_agent 코드 01

이제 08_voice_agent 코드 01 슬라이드입니다. 노트북 위치는 08_voice_agent.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

265. 08_voice_agent 코드 02

이제 08_voice_agent 코드 02 슬라이드입니다. 노트북 위치는 08_voice_agent.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입

니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

266. 08_voice_agent 코드 03

이제 08_voice_agent 코드 03 슬라이드입니다. 노트북 위치는 08_voice_agent.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent를 준비합니다. 함수 search_tool, calendar_tool는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

267. 08_voice_agent 코드 04

이제 08_voice_agent 코드 04 슬라이드입니다. 노트북 위치는 08_voice_agent.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

268. 08_voice_agent 코드 05

이제 08_voice_agent 코드 05 슬라이드입니다. 노트북 위치는 08_voice_agent.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 함수 stream_agent_response는 실제 작업 단위입니다. 핵심 API는 astream입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

269. 08_voice_agent 코드 06

이제 08_voice_agent 코드 06 슬라이드입니다. 노트북 위치는 08_voice_agent.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 weather_tool, reminder_tool는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

270. 08_voice_agent 코드 07

이제 08_voice_agent 코드 07 슬라이드입니다. 노트북 위치는 08_voice_agent.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

271. 08_voice_agent 코드 08

이제 08_voice_agent 코드 08 슬라이드입니다. 노트북 위치는 08_voice_agent.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영

코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

272. 09_production.ipynb

이제 CODE 09_production.ipynb로 넘어갑니다. 09_production.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 05_advanced/09_production.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

273. 09_production 코드 01

이제 09_production 코드 01 슬라이드입니다. 노트북 위치는 09_production.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; os를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

274. 09_production 코드 02

이제 09_production 코드 02 슬라이드입니다. 노트북 위치는 09_production.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

275. 09_production 코드 03

이제 09_production 코드 03 슬라이드입니다. 노트북 위치는 09_production.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

276. 09_production 코드 04

이제 09_production 코드 04 슬라이드입니다. 노트북 위치는 09_production.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

277. 09_production 코드 05

이제 09_production 코드 05 슬라이드입니다. 노트북 위치는 09_production.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 language_models에서 GenericFakeChatModel; messages에서 AIMessage를 준비합니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

278. 09_production 코드 06

이제 09_production 코드 06 슬라이드입니다. 노트북 위치는 09_production.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 search_tool, test_search_tool는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

279. 09_production 코드 07

이제 09_production 코드 07 슬라이드입니다. 노트북 위치는 09_production.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

280. 09_production 코드 08

이제 09_production 코드 08 슬라이드입니다. 노트북 위치는 09_production.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

281. 09_production 코드 09

이제 09_production 코드 09 슬라이드입니다. 노트북 위치는 09_production.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 json를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

282. 09_production 코드 10

이제 09_production 코드 10 슬라이드입니다. 노트북 위치는 09_production.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

283. 고급 에이전트는 실행 흐름과 상태를 설계할 때 운영 가능해집니다

고급 에이전트는 실행 흐름과 상태를 설계할 때 운영 가능해집니다. AI 에이전트 개발 고급. Middleware · Multi-agent · Memory · Production. 이 트랙에서 남겨야 할 핵심은 미들웨어, 멀티에이전트, 메모리, RAG, SQL, 데이터 분석, 음성, 평가와 배포를 운영 관점에서 결합하는 일입니다. 실습 코드를 모두 외울 필요는 없습니다. 대신 어떤 상황에서 어떤 레이어를 선택하고, 상태와 도구와 권한을 어디에서 관리할지 말로 설명하는 수준까지 가야 합니다. 이 기준으로 다시 노트북을 열어 보면, 각 코드 셀이 단순 예제가 아니라 운영 가능한 에이전트를 구성하는 하나의 결정 지점으로 보입니다.